# SIMATS ENGINEERING
## ASSESSMENT TOOL 1
## Coding Challenge

**COURSE CODE: ITA04 COURSE NAME: Statistics with R Programming**

**COURSE OUTCOME COVERED**

**CO2:** R Programming Structures, Different types of loops such as: for (), while() loops, Control Statements, Looping Over Non-Vector Sets, - If-Else, Arithmetic and Boolean Operators and values, Default Values for Argument, Return Values, Deciding Whether to explicitly call return- Returning Complex Objects, Functions are Objective, No Pointers in R, Recursion. (BL4-BL5)

Assessment Tool Weightage: 25%

**QUESTIONS: 10** **Total Marks: 50**

## Coding Challenge

***Code of Conduct:***

*I **A. Midhuna (192521302)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

*A. Midhuna*
***Signature of the student:***

# Coding Challenge

1. Write an R program to declare variables of each basic data type (numeric, integer, character, logical, complex) and print their class using class ().

   **ANSWER:**

   **Coding:**

```
# Declare variables
a <- 10.5
b <- 10L
c <- "Hello"
d <- TRUE
e <- 3 + 2i
# Print classes
print(class(a))
print(class(b))
print(class(c))
print(class(d))
print(class(e))
```

   **Output:**

```
# Declare variables
a <- 10.5
b <- 10L
c <- "Hello"
d <- TRUE
e <- 3 + 2i

# Print classes
print(class(a))
print(class(b))
print(class(c))
print(class(d))
print(class(e))
```

```
[1] "numeric"
[1] "integer"
[1] "character"
[1] "logical"
[1] "complex"
```

2. Create a list containing a vector, a matrix, and a character string. Write a program to access and print each element of the list individually.

**ANSWER:**

**Coding:**

```
# Create a list
mylist <- list (
  c (10, 20, 30),
  matrix (1:4, nrow = 2),
  "Hello R"
)
# Access and print each element
Print (mylist [[1]])
Print (mylist [[2]])
Print (mylist [[3]])
```

**Output:**

```
# Create a list
mylist <- list(
  c(10, 20, 30),
  matrix(1:4, nrow = 2),
  "Hello R"
)

# Access and print each element
print(mylist[[1]])
print(mylist[[2]])
print(mylist[[3]])
```

```
[1] 10 20 30
     [,1] [,2]
[1,]    1    3
[2,]    2    4
[1] "Hello R"
```

3. Write a program to create a 4x4 matrix of integers and perform matrix addition, subtraction, and multiplication with another 4x4 matrix.

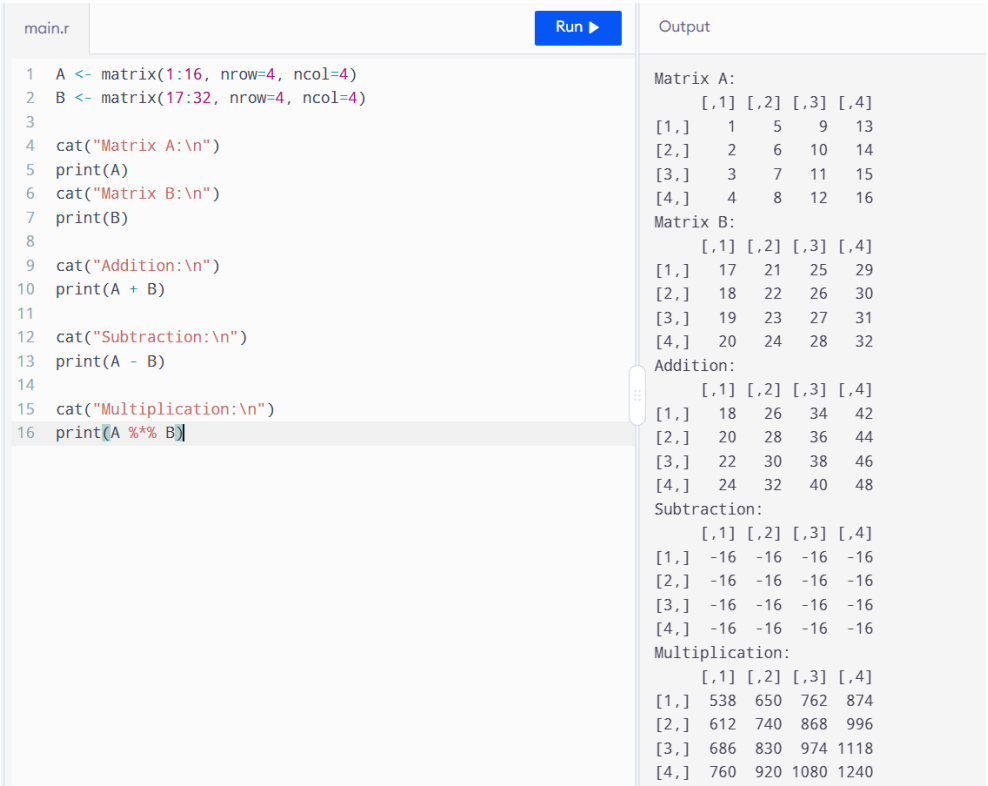
**ANSWER:**

**Coding:**

```
# Create two 4x4 integer matrices
A <- matrix (1:16, nrow = 4, ncol = 4)
B <- matrix (17:32, nrow = 4, ncol = 4)
# Display matrices
Cat ("Matrix A:\n")
print(A)
cat ("Matrix B:\n")
print(B)
# Matrix Addition
cat("Addition:\n")
print (A + B)
```

```
# Matrix Subtraction
cat("Subtraction:\n")
print (A - B)
# Matrix Multiplication
cat("Multiplication:\n")
print (A %*% B)
```

**Output**:




4. Create a 3-dimensional array of size 2x3x4 filled with random numbers, then write a program to extract and print a specific slice of the array.

**ANSWER:**

**Coding:**

```
x <- array (runif (24), dim = c (2,3,4))
print ("3D Array:")
print(x)
print ("First Slice:")
print (x [, ,1])
```

**Output:**




5. Define an S3 class called Student with fields name, age, and marks, and write a method to display student details. Then implement the same using an S4 class and compare the outputs.

**ANSWER:**

**Coding**:

```
# S3 Class
Student <- function (name, age, marks) {
  Structure (list (name=name, age=age, marks=marks),
          class="Student")
}
print. Student <- function(x) {
  print(x)
}
s1 <- Student ("Arun", 20, 85)
print(s1)
# S4 Class
setClass ("Student4",
        slots=list (name="character",
                age="numeric", marks="numeric"))
s2 <- new ("Student4", name="Arun", age=20, marks=85)
show(s2)
```

**Output:**

```
student1 <- list(name = "Ravi", age = 20, marks = 85)
class(student1) <- "Student"
print.Student <- function(x) {
  cat("S3 Student Details\n")
  cat("Name:", x$name, "\n")
  cat("Age:", x$age, "\n")
  cat("Marks:", x$marks, "\n")
}
print(student1)
setClass("StudentS4",
         slots = c(name = "character",
                   age = "numeric",
                   marks = "numeric"))
student2 <- new("StudentS4",
                name = "Ravi", age = 20, marks = 85)
show(student2)
```

```
S3 Student Details
Name: Ravi
Age: 20
Marks: 85
An object of class "StudentS4"
Slot "name":
[1] "Ravi"

Slot "age":
[1] 20

Slot "marks":
[1] 85
```

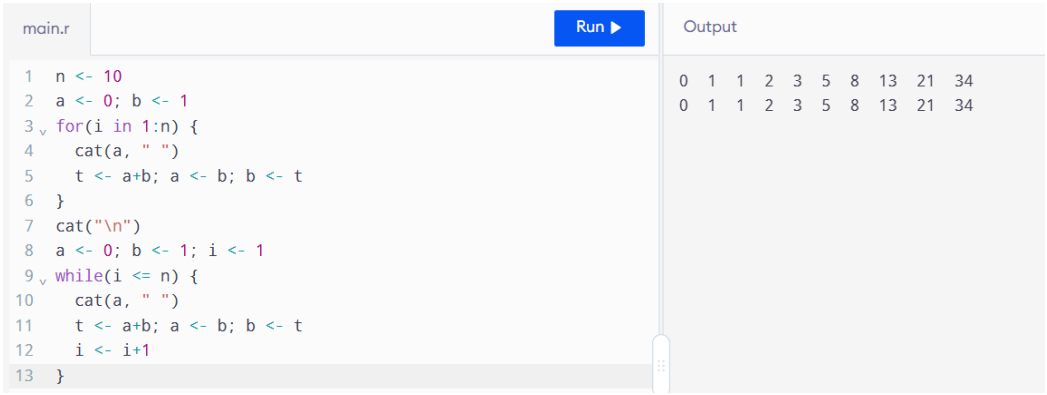
6. Write an R program using a for loop to print the Fibonacci series up to n terms, then rewrite the same using a while loop.

**ANSWER:**

**Coding:**

```
n <- 10
a <- 0; b <- 1
for (i in 1: n) {
  cat (a, " ")
  t <- a+b; a <- b; b <- t
}
cat("\n")
a <- 0; b <- 1; i <- 1
while (i <= n) {
  cat (a, " ")
  t <- a+b; a <- b; b <- t
  i <- i+1
}
```

**Output:**




7. Write a program using nested if-else statements to categorize a student's grade based on marks entered by the user.

**ANSWER:**

**Coding:**

```
marks <- 85
if (marks >= 90) {
  print ("Grade A")
} else {
  if (marks >= 75) {
    print ("Grade B")
  } else {
    if (marks >= 50) {
      print ("Grade C")
    } else {
      Print ("Grade F")
    }
  }
}
```

**Output:**



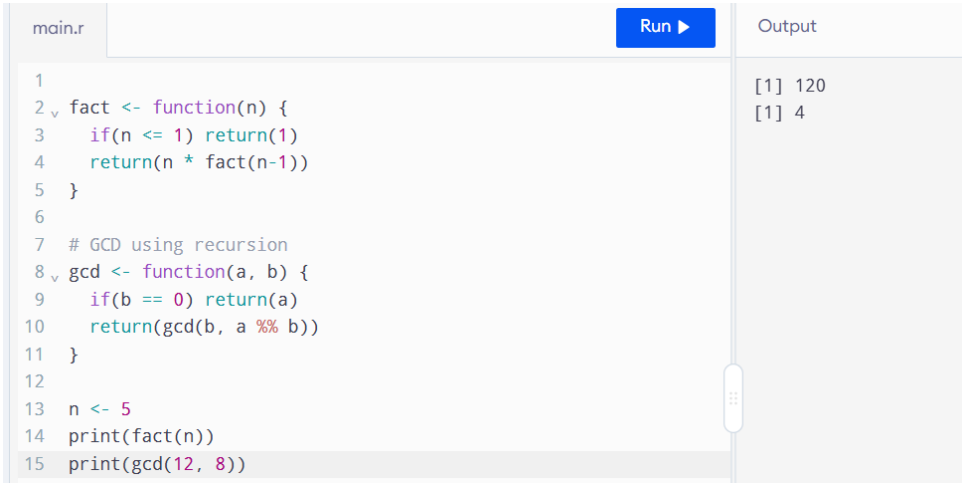

8. Write a recursive R function to calculate the factorial of a number, and another recursive function to find the GCD of two numbers.

**ANSWER:**

**Coding:**

```
fact <- function(n) {
  if (n <= 1) return (1)
  return (n * fact(n-1))
}
# GCD using recursion
gcd <- function (a, b) {
  if (b == 0) return(a)
  return (gcd (b, a %% b))
}
n <- 5
print(fact(n))
print (gcd (12, 8))
```

**Output:**




9. Write a function in R with default argument values that calculates the simple interest, and demonstrate calling it with and without passing all arguments.

**ANSWER:**

**Coding:**

```
interest <- function (p=1000, r=5, t=2) {
  si <- (p*r*t)/100
  return(si)
}
Cat ("Default Simple Interest:", interest (), "\n")
Cat ("Simple Interest:", interest (2000, 10, 3), "\n")
Cat ("Interest for 5000:", interest (5000, 6, 2), "\n")
```

**Output:**



10. Write a program to loop over a non-vector set (like a list of mixed data types) and print each element along with its data type.

**ANSWER:**

**Coding:**

```
data <- list (10, "Hello", TRUE, 25.5, "R Programming")

for (i in 1:length(data)) {
  cat ("Element:", data[[i]], "\n")
  cat ("Data type:", typeof(data[[i]]), "\n\n")
}
Cat ("All elements displayed successfully.\n")
```

**Output:**



## RUBRICS FOR CALCULATION

| Criteria | Weightage | Level 4 (Excellent) | Level 3 (Good) | Level 2 (Satisfactory) | Level 1 (Needs Improvement) |
|---|---|---|---|---|---|
| **Data Preparation** | 15% | Dataset is correctly created/imported with no errors. | Dataset is mostly correct with minor errors. | Dataset contains some inaccuracies affecting calculations. | Dataset is incorrect or incomplete. |
| **Calculation Accuracy** | 35% | All statistical measures/calculations are correct and accurately interpreted. | Most calculations are correct with minor errors. | Several calculation errors present. | Major errors; calculations are incorrect or missing. |
| **Use of R Functions** | 20% | Appropriate R functions are selected and used efficiently. | Functions are mostly appropriate with minor issues. | Limited or partially correct use of functions. | Incorrect or no use of relevant R functions. |
| **Analysis and Interpretation** | 20% | Results are clearly interpreted with logical conclusions. | Interpretation is mostly correct with minor gaps. | Limited interpretation of results. | No meaningful interpretation provided. |
| **Presentation of Results** | 10% | Output is clearly organized, labeled, and documented. | Output is organized with minor formatting issues. | Output lacks clarity or proper labeling. | Output is poorly presented or incomplete. |